比赛复现脚本 - 全部代码

文件目录：

- environment.yml (yaml)
- convert_dicom_to_nifti.py (python)
- process_masks_and_json.py (python)
- convert_test_dicom.py (python)
- custom_trainers.py (python)
- run_train.sh (bash)
- run_infer.sh (bash)

environment.yml (yaml)

```yaml
name: nnunet_env
channels:
  - pytorch
  - nvidia
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - pip
  - pip:
    - torch
    - torchvision
    - torchaudio
    - nnunetv2
    - SimpleITK
    - nibabel
    - numpy
    - scipy
    - scikit-image
    - pandas
    - matplotlib
    - tqdm
```

convert_dicom_to_nifti.py (python)

```python
# -*- coding: utf-8 -*-
"""
训练集 DICOM → 3D NIfTI 转换脚本
输出: nnUNet_raw/Dataset601_Competition/imagesTr/{id}_0000.nii.gz
用法: python convert_dicom_to_nifti.py
"""
import os
import re
import traceback
import SimpleITK as sitk

# ======================== 配置区 ========================
SRC_ROOT = "./competition_data_2026/train/images"
DST_ROOT = "./nnUNet_raw/Dataset601_Competition/imagesTr"
CHANNEL_SUFFIX = "0000"
SKIP_EXISTED = True
FIX_CT_HU = False
MIN_OUTPUT_SIZE = 1024
# ========================================================

def is_valid_nnunet_caseid(case_id):
    return bool(re.fullmatch(r'[A-Za-z0-9_]+', case_id))

def locate_dicom_dir(case_dir):
    reader = sitk.ImageSeriesReader()
    if reader.GetGDCMSeriesIDs(case_dir):
        return case_dir
    for name in sorted(os.listdir(case_dir)):
        sub_path = os.path.join(case_dir, name)
        if os.path.isdir(sub_path) and reader.GetGDCMSeriesIDs(sub_path):
            return sub_path
    return None

def fix_ct_hu_value(image):
    keys = image.GetMetaDataKeys()
    slope = 1.0
    intercept = 0.0
    if "0028|1053" in keys:
        slope = float(image.GetMetaData("0028|1053"))
    if "0028|1052" in keys:
        intercept = float(image.GetMetaData("0028|1052"))
    arr = sitk.GetArrayFromImage(image)
    arr = arr * slope + intercept
    new_img = sitk.GetImageFromArray(arr)
    new_img.CopyInformation(image)
    return new_img
```

```
def convert_one_case(case_id, case_dir):
    out_file = os.path.join(DST_ROOT, f"{case_id}_{CHANNEL_SUFFIX}.nii.gz")
    if SKIP_EXISTED and os.path.exists(out_file):
        print(f"[跳过] {case_id}: 文件已存在")
        return None
    if not is_valid_nnunet_caseid(case_id):
        print(f"[警告] {case_id}: 病例名含有非法字符")
    dicom_folder = locate_dicom_dir(case_dir)
    if dicom_folder is None:
        print(f"[跳过] {case_id}: 未找到DICOM序列")
        return False
    reader = sitk.ImageSeriesReader()
    series_id_list = reader.GetGDCMSeriesIDs(dicom_folder)
    best_slice_files = []
    for sid in series_id_list:
        file_list = reader.GetGDCMSeriesFileNames(dicom_folder, sid)
        if len(file_list) > len(best_slice_files):
            best_slice_files = file_list
    if len(series_id_list) > 1:
        print(f"[提示] {case_id}: {len(series_id_list)}个序列，选择切片最多({len(best_slice_
    reader.SetFileNames(best_slice_files)
    img_3d = reader.Execute()
    if FIX_CT_HU:
        img_3d = fix_ct_hu_value(img_3d)
    print(f"[信息] {case_id} | size:{img_3d.GetSize()} | spacing:{img_3d.GetSpacing()}")
    sitk.WriteImage(img_3d, out_file, True)
    if not os.path.exists(out_file) or os.path.getsize(out_file) < MIN_OUTPUT_SIZE:
        print(f"[失败] {case_id}: 输出文件为空")
        return False
    print(f"[成功] {case_id}: {len(best_slice_files)}张切片 → {os.path.basename(out_file)}
    return True

def main():
    os.makedirs(DST_ROOT, exist_ok=True)
    all_cases = sorted([d for d in os.listdir(SRC_ROOT) if os.path.isdir(os.path.join(SR
    print(f"共扫描到 {len(all_cases)} 个病例\n")
    success = skip = fail = 0
    for case_id in all_cases:
        try:
            res = convert_one_case(case_id, os.path.join(SRC_ROOT, case_id))
            if res is True: success += 1
            elif res is None: skip += 1
            else: fail += 1
        except Exception:
            print(f"[异常失败] {case_id}\n{traceback.format_exc()}")
            fail += 1
    print(f"\n转换结束 | 成功:{success} | 跳过:{skip} | 失败:{fail} | 总计:{len(all_cases)}"
```

```
if __name__ == "__main__":
    main()
```

process_masks_and_json.py (python)

```
# -*- coding: utf-8 -*-
"""
Mask 重采样 + dataset.json 生成脚本
输出: labelsTr/{id}.nii.gz + dataset.json
用法: python process_masks_and_json.py
"""
import os
import traceback
import json
import SimpleITK as sitk

# ======================== 配置区 ========================
IMAGE_TR_ROOT = "./nnUNet_raw/Dataset601_Competition/imagesTr"
MASK_SRC_ROOT = "./competition_data_2026/train/labels"
LABEL_TR_OUT = "./nnUNet_raw/Dataset601_Competition/labelsTr"
DATASET_JSON_PATH = "./nnUNet_raw/Dataset601_Competition/dataset.json"

LABELS = {
    "background": 0,
    "class_1": 1,
    "class_2": 2,
    "class_3": 3
}
# ========================================================

os.makedirs(LABEL_TR_OUT, exist_ok=True)

def get_ref_image(case_id):
    ref_img_path = os.path.join(IMAGE_TR_ROOT, f"{case_id}_0000.nii.gz")
    if not os.path.exists(ref_img_path):
        return None
    return sitk.ReadImage(ref_img_path)

def process_one_mask(case_id):
    ref_image = get_ref_image(case_id)
    if ref_image is None:
        print(f"[跳过] {case_id}: 找不到对应CT图像")
        return False
    mask_src_path = os.path.join(MASK_SRC_ROOT, f"{case_id}_mask.nii.gz")
    if not os.path.exists(mask_src_path):
```

```
        print(f"[跳过] {case_id}: mask源文件不存在")
        return False
    mask = sitk.ReadImage(mask_src_path)
    print(f"[信息] {case_id} 原始mask size:{mask.GetSize()}, spacing:{mask.GetSpacing()}"
    resampler = sitk.ResampleImageFilter()
    resampler.SetReferenceImage(ref_image)
    resampler.SetInterpolator(sitk.sitkNearestNeighbor)
    mask_resampled = resampler.Execute(mask)
    out_mask_path = os.path.join(LABEL_TR_OUT, f"{case_id}.nii.gz")
    sitk.WriteImage(mask_resampled, out_mask_path, True)
    print(f"[完成] {case_id} → {case_id}.nii.gz | size:{mask_resampled.GetSize()}")
    return True

def generate_dataset_json(case_id_list):
    dataset = {
        "name": "Competition",
        "description": "competition ct segmentation",
        "reference": "none",
        "licence": "none",
        "release": "1.0",
        "modality": {"0": "CT"},
        "channel_names": {"0": "CT"},
        "labels": LABELS,
        "numTraining": len(case_id_list),
        "file_ending": ".nii.gz",
        "training": [
            {"image": f"./imagesTr/{cid}_0000.nii.gz", "label": f"./labelsTr/{cid}.nii.g
            for cid in case_id_list
        ]
    }
    with open(DATASET_JSON_PATH, "w", encoding="utf-8") as f:
        json.dump(dataset, f, indent=2, ensure_ascii=False)
    print(f"\ndataset.json 已生成，共 {len(case_id_list)} 套训练数据")

def main():
    suffix = "_0000.nii.gz"
    all_cases = sorted([
        fname[:-len(suffix)]
        for fname in os.listdir(IMAGE_TR_ROOT)
        if os.path.isfile(os.path.join(IMAGE_TR_ROOT, fname)) and fname.endswith(suffix)
    ])
    print(f"待处理病例总数: {len(all_cases)}\n")
    ok_cnt = 0
    valid_case_list = []
    for cid in all_cases:
        try:
            if process_one_mask(cid):
                ok_cnt += 1
                valid_case_list.append(cid)
        except Exception:
```

```
            print(f"[mask失败] {cid}:\n{traceback.format_exc()}")
    print(f"\nmask处理结束: 成功 {ok_cnt}/{len(all_cases)}")
    generate_dataset_json(valid_case_list)

if __name__ == "__main__":
    main()
```

convert_test_dicom.py (python)

```
# -*- coding: utf-8 -*-
"""
测试集 DICOM → 3D NIfTI 转换脚本
输出: nnUNet_raw/Dataset601_Competition/imagesTs/{id}_0000.nii.gz
用法: python convert_test_dicom.py
"""
import os
import traceback
import SimpleITK as sitk

# ======================== 配置区 ========================
SRC_ROOT = "./competition_data_2026/test/images"
DST_ROOT = "./nnUNet_raw/Dataset601_Competition/imagesTs"
CHANNEL_SUFFIX = "0000"
SKIP_EXISTED = True
# ========================================================

def locate_dicom_dir(case_dir):
    reader = sitk.ImageSeriesReader()
    if reader.GetGDCMSeriesIDs(case_dir):
        return case_dir
    for name in sorted(os.listdir(case_dir)):
        sub = os.path.join(case_dir, name)
        if os.path.isdir(sub) and reader.GetGDCMSeriesIDs(sub):
            return sub
    return None

def convert_one_case(case_id, case_dir):
    out_name = f"{case_id}_{CHANNEL_SUFFIX}.nii.gz"
    out_path = os.path.join(DST_ROOT, out_name)
    if SKIP_EXISTED and os.path.exists(out_path):
        print(f"[跳过-已存在] {case_id}")
        return True
    dicom_dir = locate_dicom_dir(case_dir)
    if dicom_dir is None:
        print(f"[跳过-无DICOM] {case_id}")
```

```
        return False
    reader = sitk.ImageSeriesReader()
    series_ids = reader.GetGDCMSeriesIDs(dicom_dir)
    best_files = []
    for sid in series_ids:
        files = reader.GetGDCMSeriesFileNames(dicom_dir, sid)
        if len(files) > len(best_files):
            best_files = files
    if len(series_ids) > 1:
        print(f"[提示] {case_id}: {len(series_ids)}个序列，选取切片最多({len(best_files)})")
    reader.SetFileNames(best_files)
    image = reader.Execute()
    sitk.WriteImage(image, out_path, True)
    print(f"[完成] {case_id}: slices={len(best_files)}, size={image.GetSize()}, spacing=
    return True

def main():
    os.makedirs(DST_ROOT, exist_ok=True)
    case_ids = sorted(d for d in os.listdir(SRC_ROOT) if os.path.isdir(os.path.join(SRC_
    print(f"共扫描到 {len(case_ids)} 个测试病例\n")
    ok_cnt = 0
    fail_list = []
    skip_no_dicom = []
    for cid in case_ids:
        try:
            success = convert_one_case(cid, os.path.join(SRC_ROOT, cid))
            if success:
                ok_cnt += 1
            else:
                skip_no_dicom.append(cid)
        except Exception:
            fail_list.append(cid)
            print(f"[失败] {cid}:\n{traceback.format_exc()}")
    print("\n======转换汇总======")
    print(f"总病例数: {len(case_ids)}")
    print(f"成功/跳过: {ok_cnt}")
    print(f"无DICOM跳过: {len(skip_no_dicom)} → {skip_no_dicom}")
    print(f"转换失败: {len(fail_list)} → {fail_list}")

if __name__ == "__main__":
    main()
```

custom_trainers.py (python)

```
# -*- coding: utf-8 -*-
"""
自定义 Trainer：修改学习率和保存周期
使用方式：nnUNetv2_train 601 3d_fullres 0 -tr nnUNetTrainer_LR005_Save50
将本文件放到 nnUNet 的 Trainer 目录下：
nnunetv2/training/nnUNetTrainer/
"""
import torch
from nnunetv2.training.nnUNetTrainer.nnUNetTrainer import nnUNetTrainer


class nnUNetTrainer_LR01_Save50(nnUNetTrainer):
    """学习率 0.01，每50epoch保存"""
    def __init__(self, plans, configuration, fold, dataset_json, device=torch.device("cu
        super().__init__(plans, configuration, fold, dataset_json, device)
        self.initial_lr = 0.01
        self.save_every = 50


class nnUNetTrainer_LR005_Save50(nnUNetTrainer):
    """学习率 0.005，每50epoch保存"""
    def __init__(self, plans, configuration, fold, dataset_json, device=torch.device("cu
        super().__init__(plans, configuration, fold, dataset_json, device)
        self.initial_lr = 0.005
        self.save_every = 50


class nnUNetTrainer_LR02_Save50(nnUNetTrainer):
    """学习率 0.02，每50epoch保存"""
    def __init__(self, plans, configuration, fold, dataset_json, device=torch.device("cu
        super().__init__(plans, configuration, fold, dataset_json, device)
        self.initial_lr = 0.02
        self.save_every = 50
```

run_train.sh (bash)

```
#!/bin/bash
# ============================================================
# 一键训练脚本
# 数据集：601（Dataset601_Competition）
# 配置：3d_fullres，fold=0，初始学习率0.01，每50epoch保存
# 使用方式：bash run_train.sh
# ============================================================
```

```
set -e

# 切换到脚本所在目录（确保相对路径正确）
cd "$(dirname "$0")"

# 设置 nnUNet 环境变量
export nnUNet_raw="./nnUNet_raw"
export nnUNet_preprocessed="./nnUNet_preprocessed"
export nnUNet_results="./nnUNet_results"

# 容器适配：/dev/shm 仅64MB，必须单进程数据加载
export TMPDIR=/tmp
export nnUNet_n_proc_DA=0
export nnUNet_compile=False
export OMP_NUM_THREADS=4
export PYTHONUNBUFFERED=1

echo "========================================"
echo "  nnUNet 训练启动"
echo "  Dataset: 601"
echo "  Config:  3d_fullres"
echo "  Fold:    0"
echo "  LR:      0.01"
echo "========================================"
echo ""

# 启动训练（中断后重新运行会自动从 checkpoint_latest.pth 续训）
nnUNetv2_train 601 3d_fullres 0

echo ""
echo "========================================"
echo "  训练完成！"
echo "  权重目录: ./nnUNet_results/Dataset601_Competition/nnUNetTrainer__nnUNetPlans__3d_
echo "========================================"
```

run_infer.sh (bash)

```
#!/bin/bash
# ============================================================
# 一键推理脚本
# 数据集：601（Dataset601_Competition）
# 配置：3d_fullres，fold=0
# 使用方式：bash run_infer.sh
# ============================================================
```

```
set -e

# 切换到脚本所在目录
cd "$(dirname "$0")"

# 设置 nnUNet 环境变量
export nnUNet_raw="./nnUNet_raw"
export nnUNet_preprocessed="./nnUNet_preprocessed"
export nnUNet_results="./nnUNet_results"

# 推理参数
INPUT_DIR="./nnUNet_raw/Dataset601_Competition/imagesTs"
OUTPUT_DIR="./predict_output"
DATASET_ID=601
CONFIG="3d_fullres"
FOLD=0

echo "========================================="
echo "  nnUNet 推理启动"
echo "  输入目录: $INPUT_DIR"
echo "  输出目录: $OUTPUT_DIR"
echo "  Dataset:  $DATASET_ID"
echo "  Config:   $CONFIG"
echo "  Fold:     $FOLD"
echo "========================================="
echo ""

# 创建输出目录
mkdir -p "$OUTPUT_DIR"

# 执行推理
nnUNetv2_predict \
  -i "$INPUT_DIR" \
  -o "$OUTPUT_DIR" \
  -d "$DATASET_ID" \
  -c "$CONFIG" \
  -f "$FOLD"

echo ""
echo "========================================="
echo "  推理完成! "
echo "  结果目录: $OUTPUT_DIR"
echo "========================================="

# 自动重命名为比赛提交格式 {id}_mask.nii.gz
echo ""
echo "正在重命名为比赛提交格式..."
cd "$OUTPUT_DIR"
for f in *.nii.gz; do
    if [[ "$f" != *_mask.nii.gz ]]; then
```

```
        newname="${f%.nii.gz}_mask.nii.gz"
        mv "$f" "$newname"
        echo "  $f -> $newname"
    fi
done
echo ""
echo "重命名完成，可直接提交！"
```